

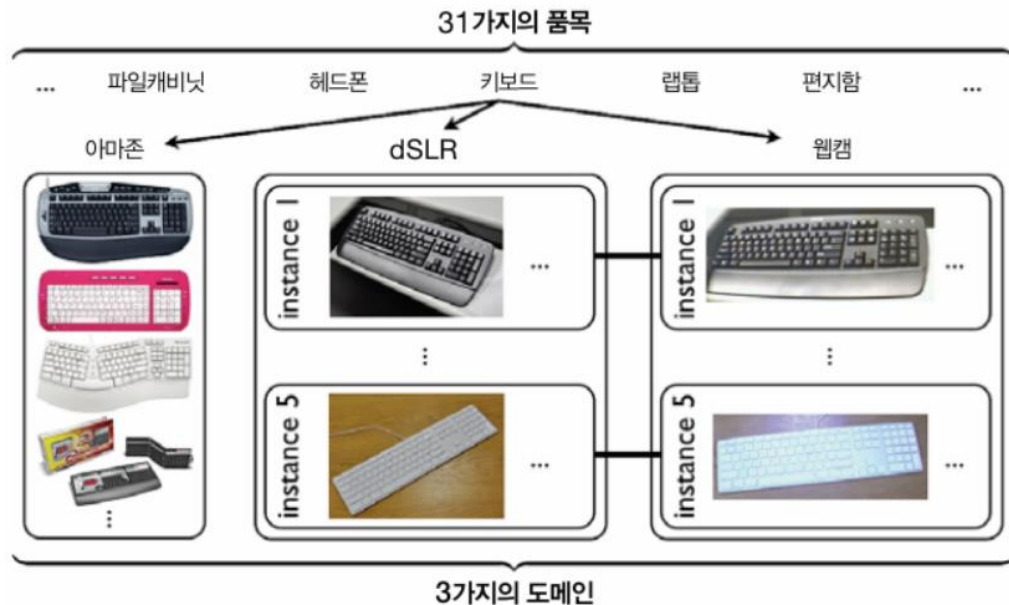
6장 : 복합 출력의 처리 방법

- 오피스 31 다차원 분류 신경망

파이썬 날코딩으로 알고 짜는 딥러닝 238p

Contents

1. 오피스31 데이터셋과 다차원 분류 3p
2. 딥러닝에서의 복합 출력의 학습법 5p
3. 아담 알고리즘 10p
4. 구현하기: 아담 모델 클래스 12p
5. 구현하기: 오피스31 데이터셋 클래스 19p
6. 실행하기 27p

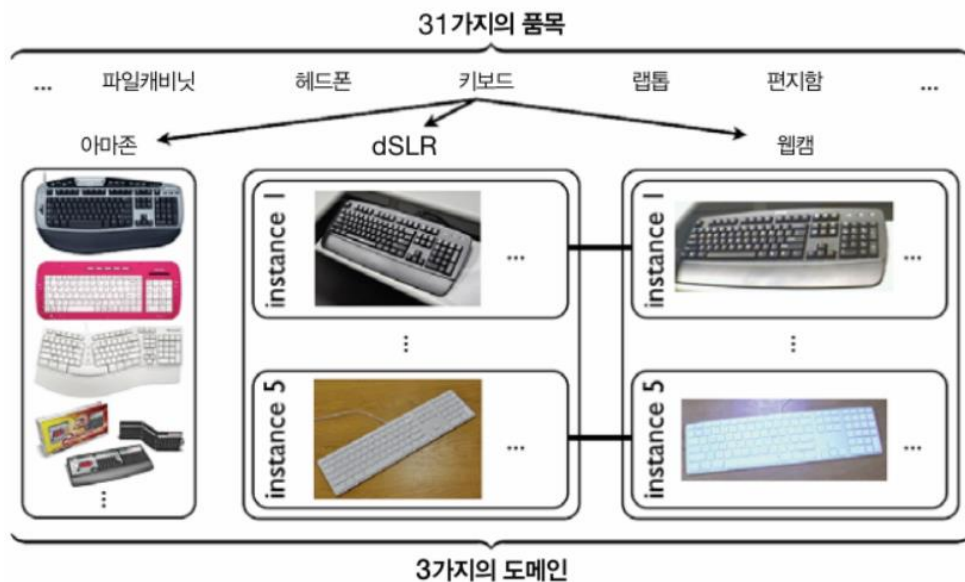


<오피스 31 데이터셋>

사무용품 이미지 4,652장으로 구성

사진 수집 방법에 따른 3가지 도메인
사진 속에 담긴 내용이 31가지 품목 중 어디에 속하는지 이중으로 레이블링

주제 : 도메인에 대한 분류와 품목에 대한 분류를 동시에 수행하는 다차원 분류 기능 모델 만들기



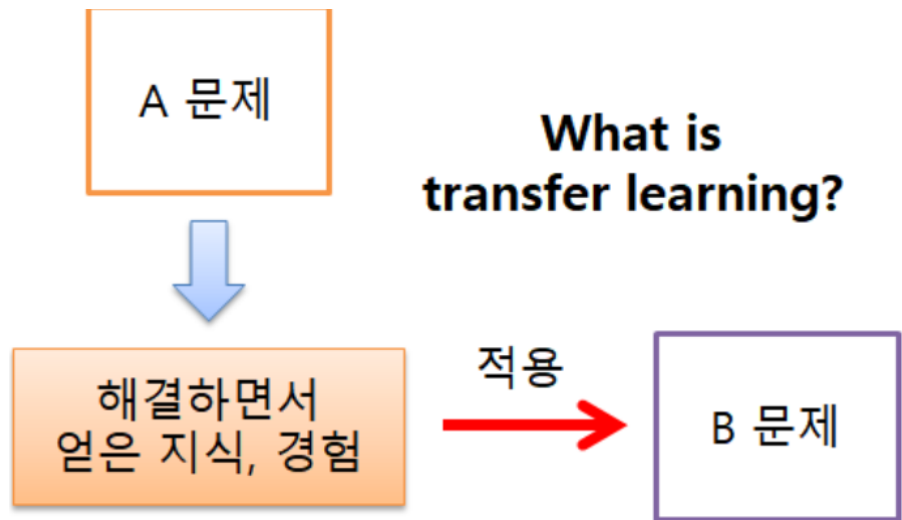
<오피스 31 데이터셋 구성>

도메인	Amazon : 웹에서 수집된 이미지
	dsLR : 디지털 사진기로 촬영한 이미지
	Webcam : 웹캠으로 촬영한 이미지
품목	배낭, 자전거, 자전거용 헬멧, 파일캐비닛 등 31가지

전이학습 vs 다차원 분류

● 전이 학습

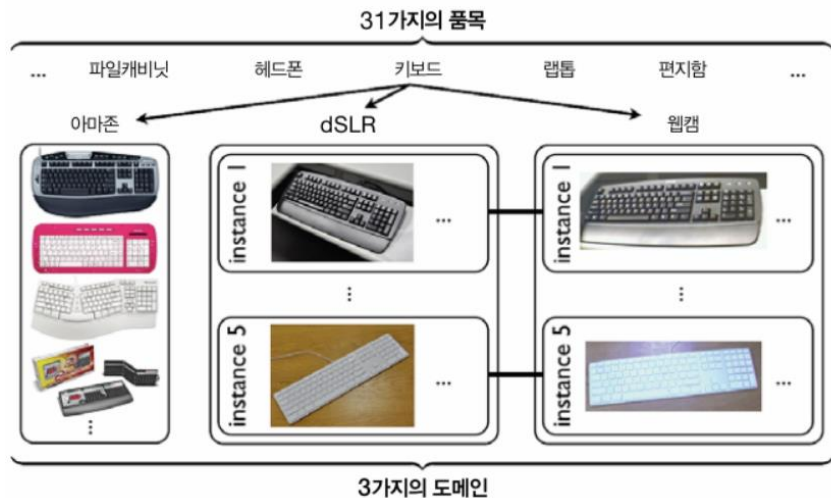
- 한 도메인에서 학습시킨 결과를 다른 도메인에 활용하는 기법
- 고비용 작업인 레이블링을 일부 도메인에서라도 줄여보려는 취지



< 전이 학습 >

● 다차원 분류

- 도메인과 품목을 동시에 판별해주는 이중 선택 분류
- 다차원 분류 대신 차원 축소를 이용한 단순 선택 분류도 가능
- 출력 후보 수 기하급수적 증가
- 도메인 특성과 품목 특성 뒤섞여 학습 성과가 좋지 않을 수 있음



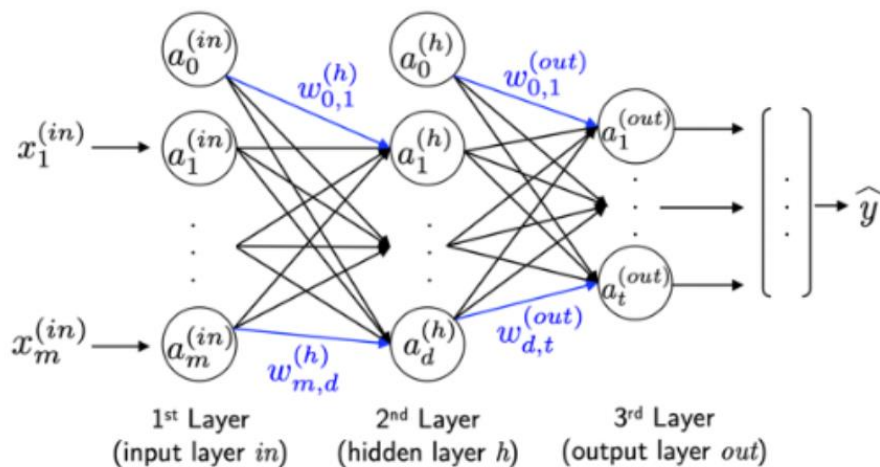
(도메인, 품목) 순서쌍을 단위로 카테고리 설정 : (아마존, 배낭), (dsLR, 배낭).



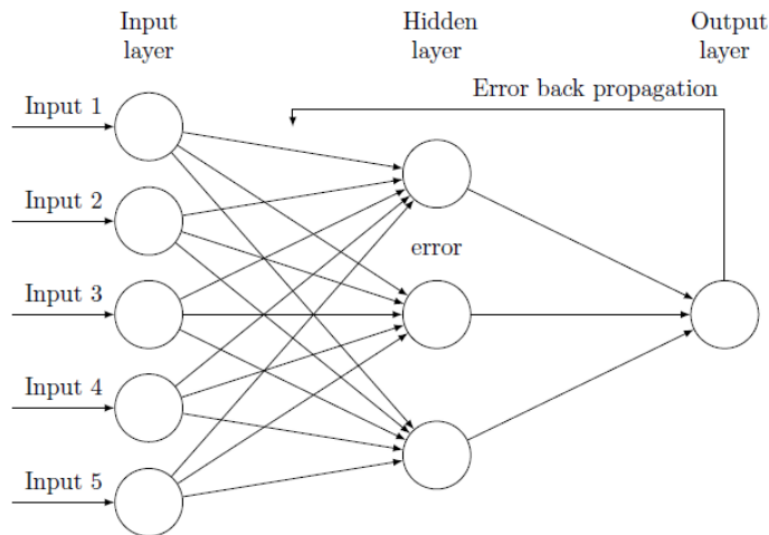
93(=3*31) 가지의 순서쌍 중 하나를 고르는 선택 분류 문제로 변함

후처리 과정의 구성

- **순전파** : 신경망 순전파 처리 결과로부터 손실 함수 계산
- **역전파** : 신경망 출력 성분별 손실 기울기 구해 신경망 역전파에 전달

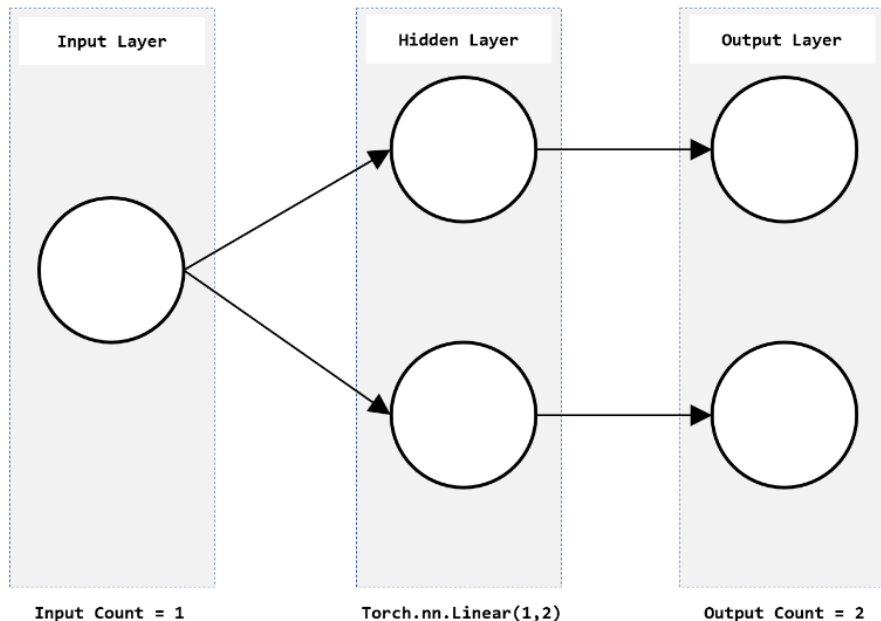


< 순전파 과정 >



< 역전파 과정 >

복합출력은 후처리를 어떻게 할지가 문제이다!



<오피스 31 데이터셋>

입력 벡터 크기

: 이미지 픽셀 수만큼의 입력 벡터 크기를 가짐

출력 벡터 크기

: 도메인 판별에 사용될 성분 3가지

+

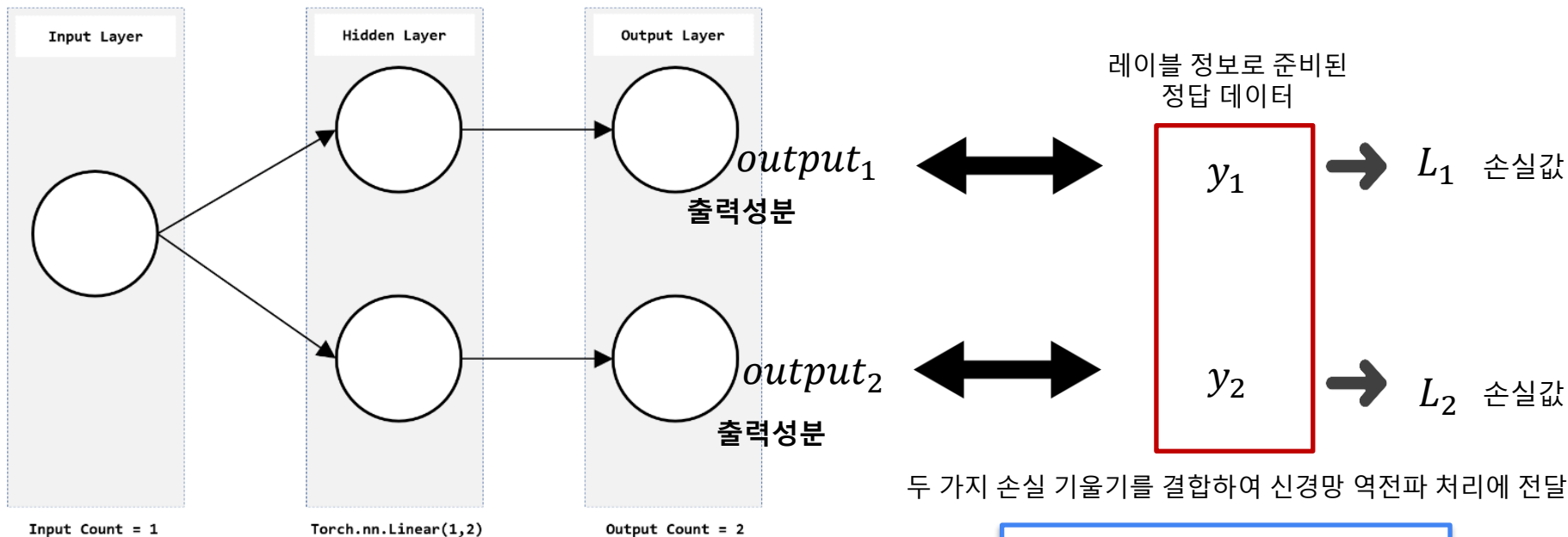
품목 판별에 사용될 성분 31가지

= 출력 벡터 크기 34가지로 키움

출력 계층 퍼셉트론 : 출력 요소 별로 역할이 분리됨

은닉 계층 퍼셉트론 : 전체 출력에 연결되어 모두에 영향을 미침

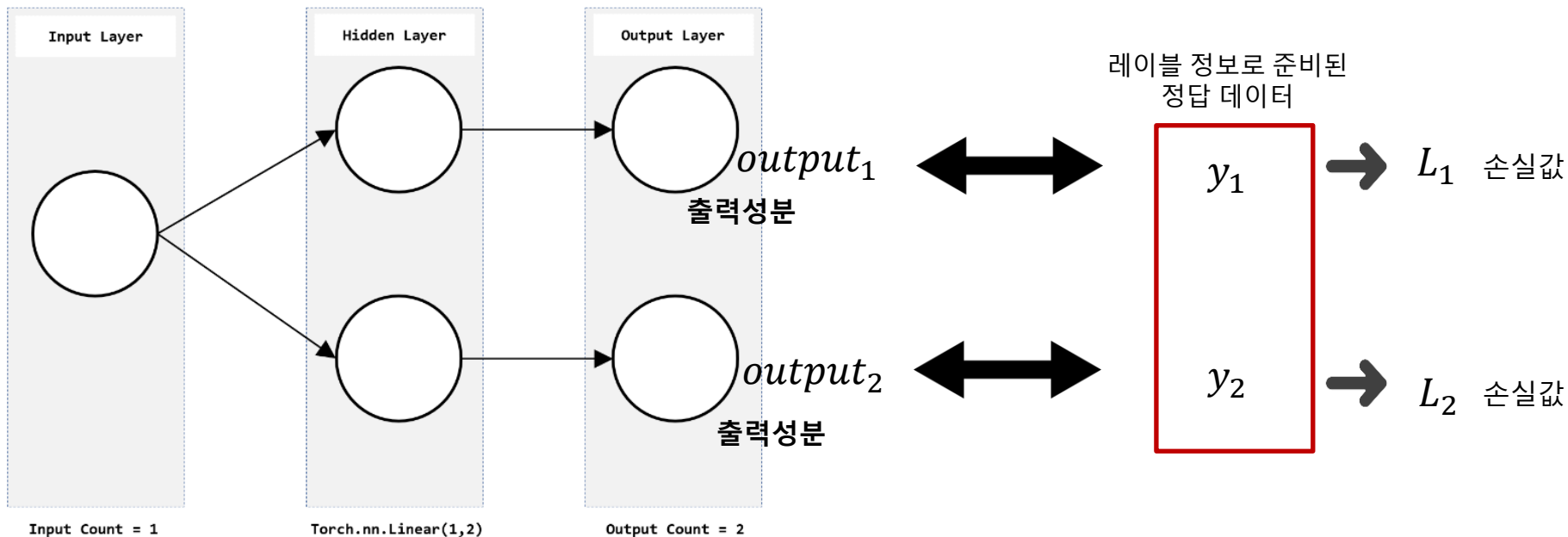
< 순전파에서의 손실 함수 계산 방법 >



< 전체 손실 함수 >

$$L = L_1 + L_2$$

< 역전파에서의 손실 함수 계산 방법 >



- Output1의 손실 기울기는 y_1 과의 후처리 과정을 통해 L_1 을 이용해 구한다.
- Output2의 손실 기울기는 y_2 와의 후처리 과정을 통해 L_2 를 이용해 구한다.

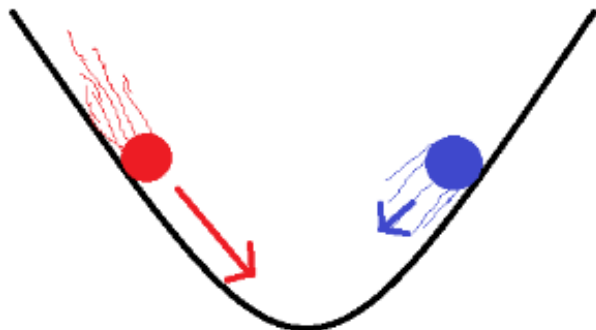
아담 알고리즘

● 모멘텀

- 파라미터 값의 최근 변화 추세를 나타내는 정보

● 아담 알고리즘

- 모멘텀과 2차 모멘텀 정보 개념 도입해 경사하강법 동작 보완
- 파라미터에 적용되는 실질적 학습률을 개별 파라미터 별로 동적 조절
- 경사하강법의 동작을 보완하고 학습 품질을 높여주는 방법



< 모멘텀 >

< 모멘텀 방식이란? >

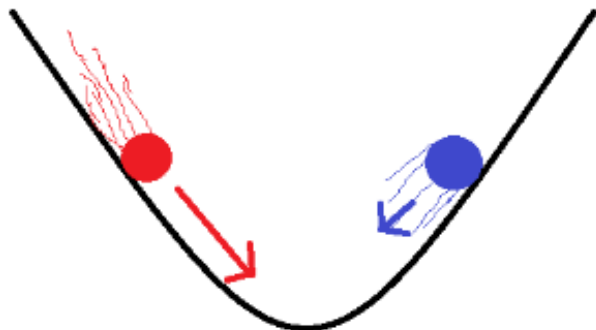
최솟값을 향해 이동하는 방향과는 별개로, 과거에 이동했던 방식을 현재 기울기를 통해 기억하면서 그 방향으로 일정 정도를 추가적으로 이동하는 방식

-> 최근의 파라미터값의 변화 추세를 나타내는 정보

아담 알고리즘

● 역전파 처리 과정 보완

- 경사하강법 : 손실 기울기 $\times$ 학습률을 각 파라미터에서 감산
- 아담 알고리즘 : 손실 기울기 대신 모멘텀 정보 이용해 보정된 값 이용
적절한 학습률 찾는 부담을 줄여주면서 학습 성능을 향상시킴



< 모멘텀 >

< 모멘텀 방식이란? >

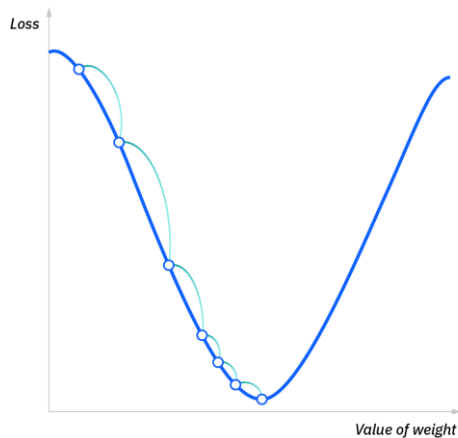
최솟값을 향해 이동하는 방향과는 별개로, 과거에 이동했던 방식을 현재 기울기를 통해 기억하면서 그 방향으로 일정 정도를 추가적으로 이동하는 방식

-> 최근의 파라미터값의 변화 추세를 나타내는 정보

경사하강법 (Gradient Descent)

- 적절한 step size가 중요함

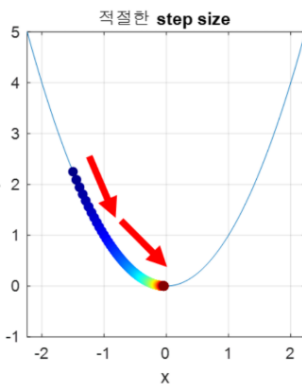
Small learning rate



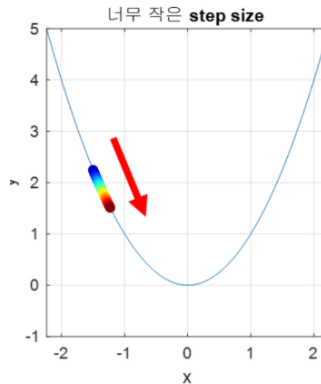
Gradient Descent Algorithm (기존의 경사 하강법)

$$w = w - \alpha * \frac{d}{dw} * J(w, b)$$

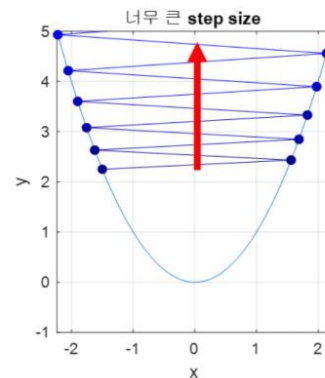
$$b = b - \alpha * \frac{d}{db} * J(w, b) \quad \alpha : \text{learning rate}$$



Step size의 크기가 너무 작게 설정되었음.



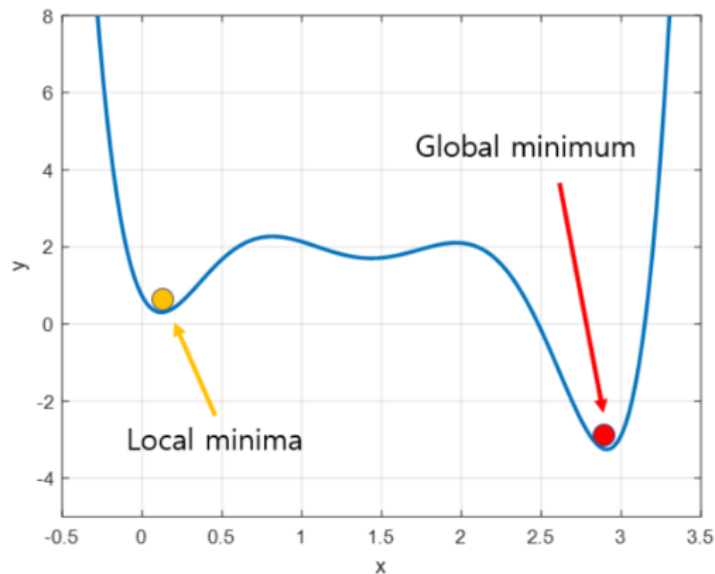
Step size의 크기가 너무 작게 설정되었음.



Step size의 크기가 과도하게 설정되었음.

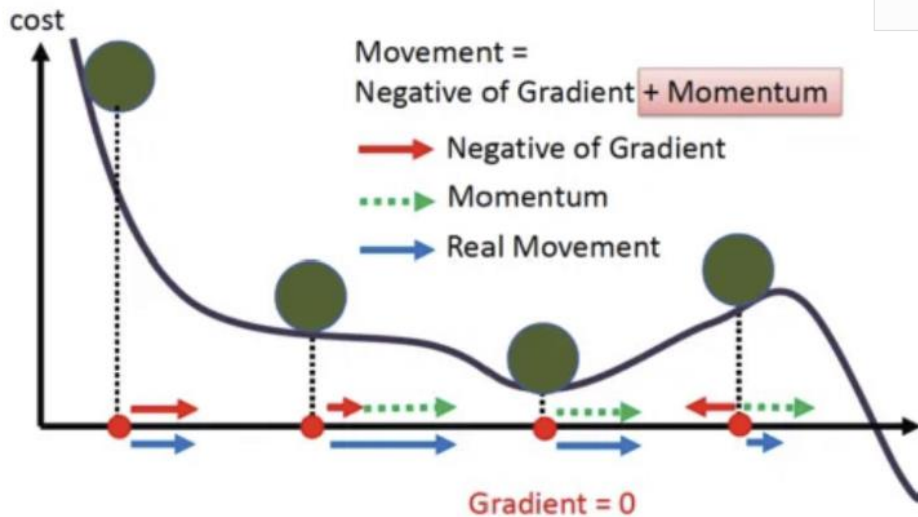
경사하강법 (Gradient Descent)

- Local Minima 문제



손실 함수에서 최적의 파라미터를 위한 Global minimum을 찾아야 한다.

Gradient Descent with Momentum



Gradient Descent with Momentum

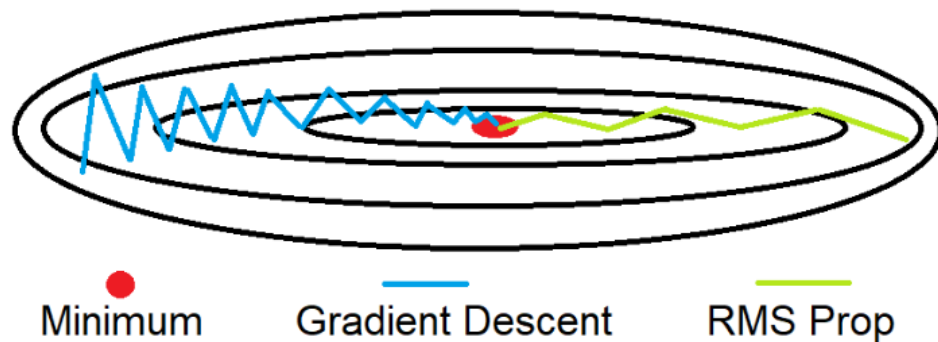
$$v_{dw} = \beta * v_{dw} + (1 - \beta) * dw \quad | \text{이때, } dw = \frac{d}{dw} * J(w, b)$$

$$v_{db} = \beta * v_{db} + (1 - \beta) * db \quad | \text{이때, } db = \frac{d}{db} * J(w, b)$$

$$w = w - \alpha * v_{dw}$$

$$b = b - \alpha * v_{db}$$

RMSprop (Root Mean Square Propagation)



RMSprop Algorithm

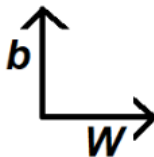
$$s_{dw} = \beta_2 * s_{dw} + (1 - \beta_2) * dw^2$$

$$s_{db} = \beta_2 * s_{db} + (1 - \beta_2) * db^2$$

$$w := w - \alpha * \frac{dw}{(\sqrt{s_{dw}} + \epsilon)}$$

$$b := b - \alpha * \frac{db}{(\sqrt{s_{db}} + \epsilon)}$$

상황	Gradient 크기	분모 ($\sqrt{s}$)	학습률 효과	목적
너무 큼	큼	큼	줄어들	발산 방지
너무 작음	작음	작음	커짐	학습 가속



Adam (Adaptive Moment Estimation)

- Momentum과 RMSprop를 섞어놓은 최적화 모델

1). 먼저 초기화를 진행하고, Momentum과 RMSprop에서 사용한 v 와 S 를 지정해준다.

$$v_{dw} = 0, s_{dw} = 0, v_{db} = 0, s_{db} = 0$$

$$v_{dw} = \beta * v_{dw} + (1 - \beta) * dw$$

$$v_{db} = \beta * v_{db} + (1 - \beta) * db$$

$$s_{dw} = \beta_2 * s_{dw} + (1 - \beta_2) * dw^2$$

$$s_{db} = \beta_2 * s_{db} + (1 - \beta_2) * db^2$$

3) Momentum과 RMSprop을 모두 사용하여 가중치 업데이트를 진행한다.

$$w := w - \alpha * \frac{v_{dw}^{biascorr}}{\sqrt{s_{dw}^{biascorr} + \epsilon}}$$

$$b := b - \alpha * \frac{v_{db}^{biascorr}}{\sqrt{s_{db}^{biascorr} + \epsilon}}$$

2). 편향 보정(Bias Correction)도 추가해준다.

$$v_{dw}^{biascorr} = \frac{v_{dw}}{(1 - \beta^t)}$$

$$v_{db}^{biascorr} = \frac{v_{db}}{(1 - \beta^t)}$$

$$s_{dw}^{biascorr} = \frac{s_{dw}}{(1 - \beta^t)}$$

$$s_{db}^{biascorr} = \frac{s_{db}}{(1 - \beta^t)}$$

기능	효과	설명
Momentum 반영	방향성 유지	진동 줄이고 빠르게 이동
RMSprop 반영	학습률 자동 조절	발산 방지 + 학습 가속
편향 보정	초기 안정화	초반 작은 값들 보정
파라미터마다 개별 학습률	복잡한 문제에도 강함	각 weight에 맞춤형 업데이트